Interpreting Training Aspects of Deep-Learned Error-Correcting Codes

https://arxiv.org/abs/2305.04347

Abhijeet Mulgund

University of Illinois Chicago

June 30, 2023







Natasha Devroye



György Turán



Miloš Žefran



Abhijeet Mulgund



Raj Shekhar

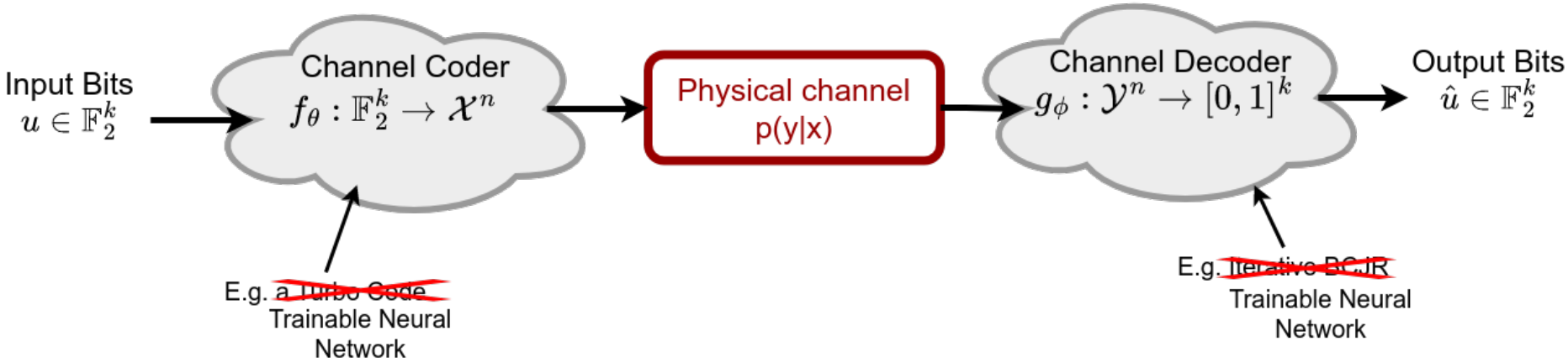


Yingyao Zhou

Outline

1. What are Deep-Learned Error Correcting Codes (DL-ECCs)?
2. Post-Hoc Interpretations
3. Interpreting the Training Process of DL-ECCs
4. Conclusions and Further Work

What are DL-ECCs?



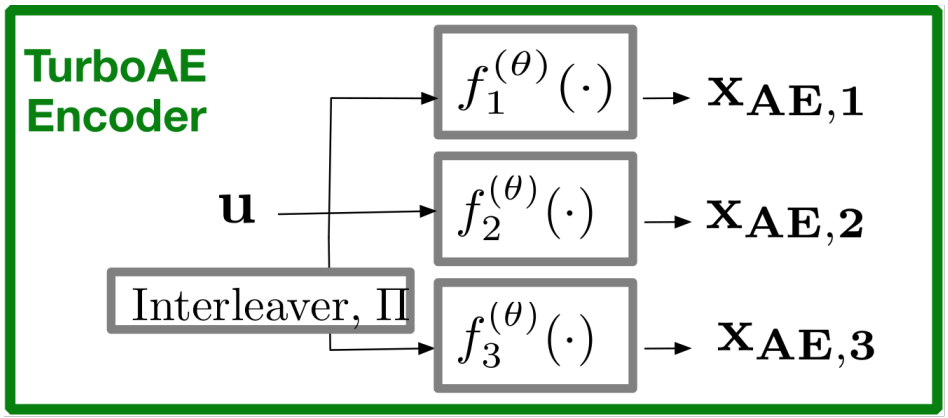
- TurboAE: *Jiang et al.* '19
- DeepCode: *Kim et al.* '20 and GBAF: Ozfatura et al. '22 - incorporates channel feedback
- Ko codes: *Makkuva et al.* '21

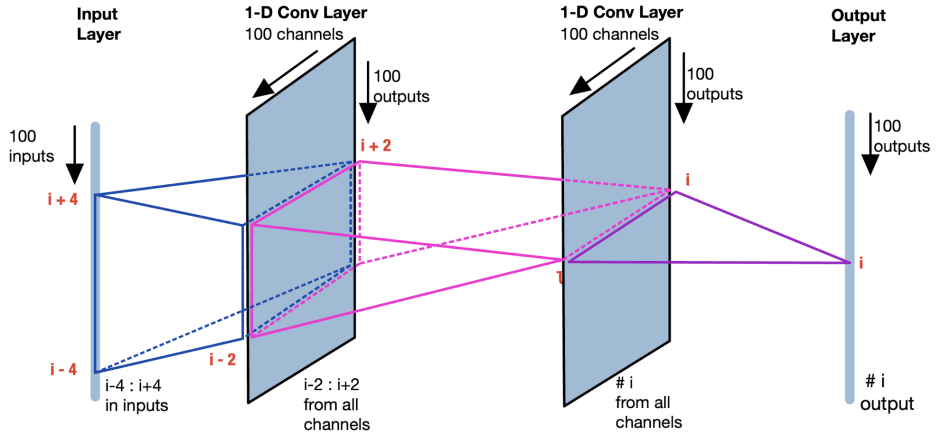
How can we interpret these black boxes?

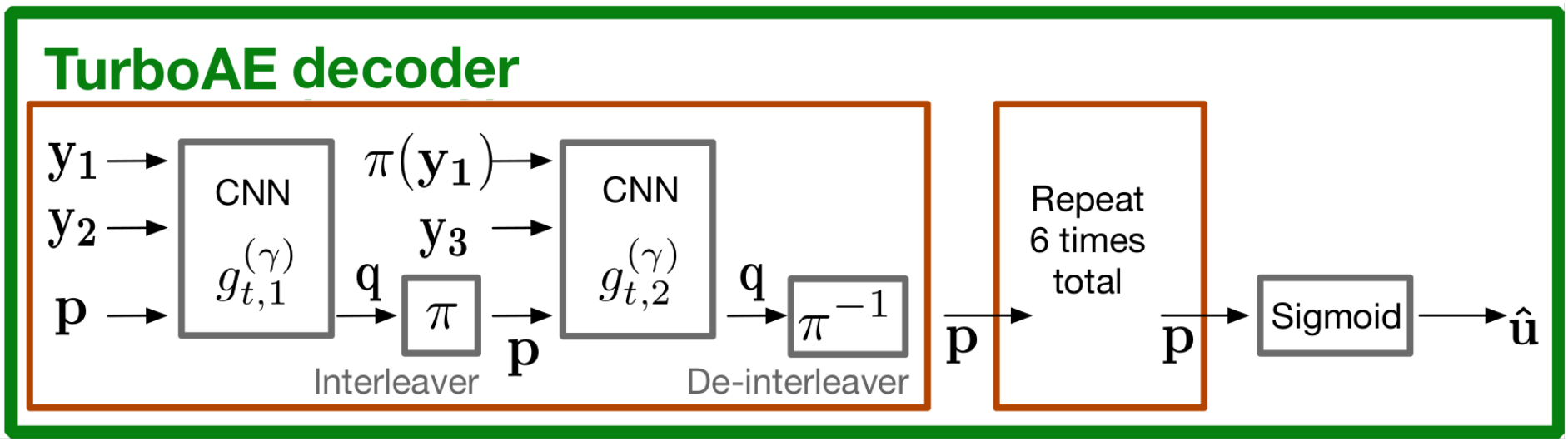
Value in Interpretations

1. Convert DL-ECCs to well-known codes with theory and efficient implementations - *Post-hoc Interpretation*
   - Exact Conversion
   - Approximate Conversion
2. Understand the training process to further improve DL-ECC and understand what Deep Learning brings to coding.
   - complementary to broader general deep learning questions of implicit regularization, DL learnability of problems, and loss landscapes.
3. DL-ECCs form a mathematically well-defined optimization problem to use as a sandbox for developing general purpose interpretability techniques.

TurboAE





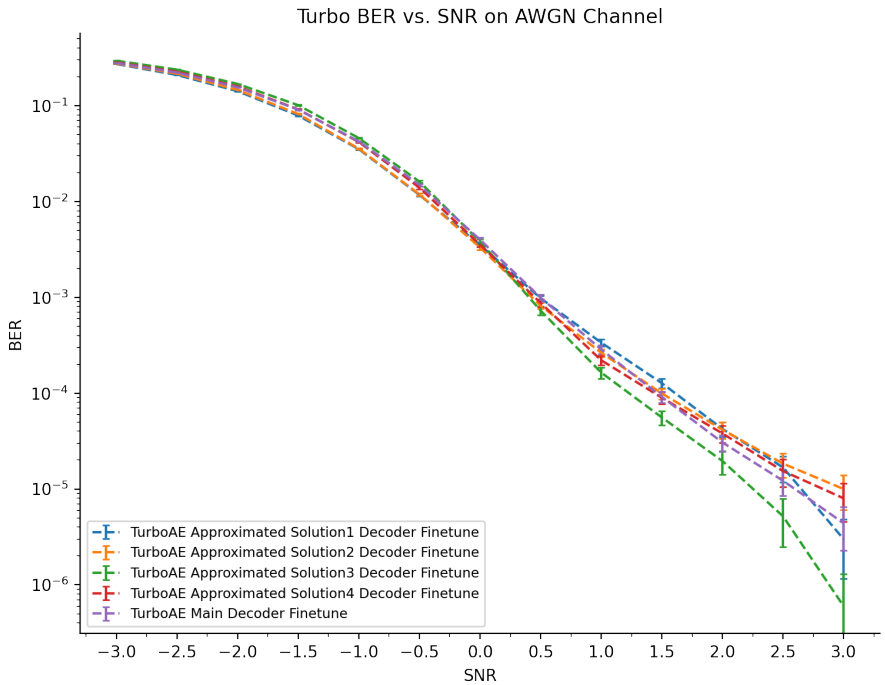


[1] Figures adapted from *Jiang et al., NeurIPS '19* and *Mulgund et al., Allerton '22*

Exact and Approximate Conversions of TurboAE Encoder (Binary)

Block #	Expression for output #$i \in \{3, 4, \cdots, 98\}$
1	$1 \oplus u_1 \oplus \bar{u}_2 \oplus u_3 \oplus \bar{u}_4 \oplus u_5$ $\oplus\ \bar{u}_2 u_3 \bar{u}_4 \oplus u_1 \bar{u}_2 u_3 \bar{u}_4 u_5$
2	$u_1 \oplus u_3 \oplus u_4 \oplus u_5$
3	$u_1 \oplus u_2 \oplus u_4 \oplus \bar{u}_3 \bar{u}_5$
where	$u_1 = x_{i+2}$, $u_2 = x_{i+1}$, $u_3 = x_i$, $u_4 = x_{i-1}$, $u_5 = x_{i-2}$, $x =$ inputs

Block #	Approximate expressions for output #i
1	$1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4 \oplus u_5$
2	$u_1 \oplus u_3 \oplus u_4 \oplus u_5$
3	Solution 1: $u_1 \oplus u_2 \oplus u_4$ Solution 2: $1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4$ Solution 3: $1 \oplus u_1 \oplus u_2 \oplus u_4 \oplus u_5$ Solution 4: $1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4 \oplus u_5$
where	$u_1 = x_{i+2}$, $u_2 = x_{i+1}$, $u_3 = x_j$, $u_4 = x_{i-1}$, $u_5 = x_{i-2}$, $x =$ inputs



[1] Figures from *Devroye et al., ISIT '23* and *Mulgund et al., Allerton '22*

Approximating using Fourier-Hadamard Expansion

Recall for $w \in \mathbb{N}$, we can write any function $f : \{-1, 1\}^w \to \mathbb{R}$ in terms of its Fourier-Hadamard expansion

$$f(x) = \sum_{S \subseteq [w]} \hat{f}(S)\chi_S(x)$$

where

- $\chi_S(x) = \prod_{i \in S} x_i$
- $\hat{f}(S) \in \mathbb{R}$

Note

- each χ_S corresponds to parity function over the indices in S.

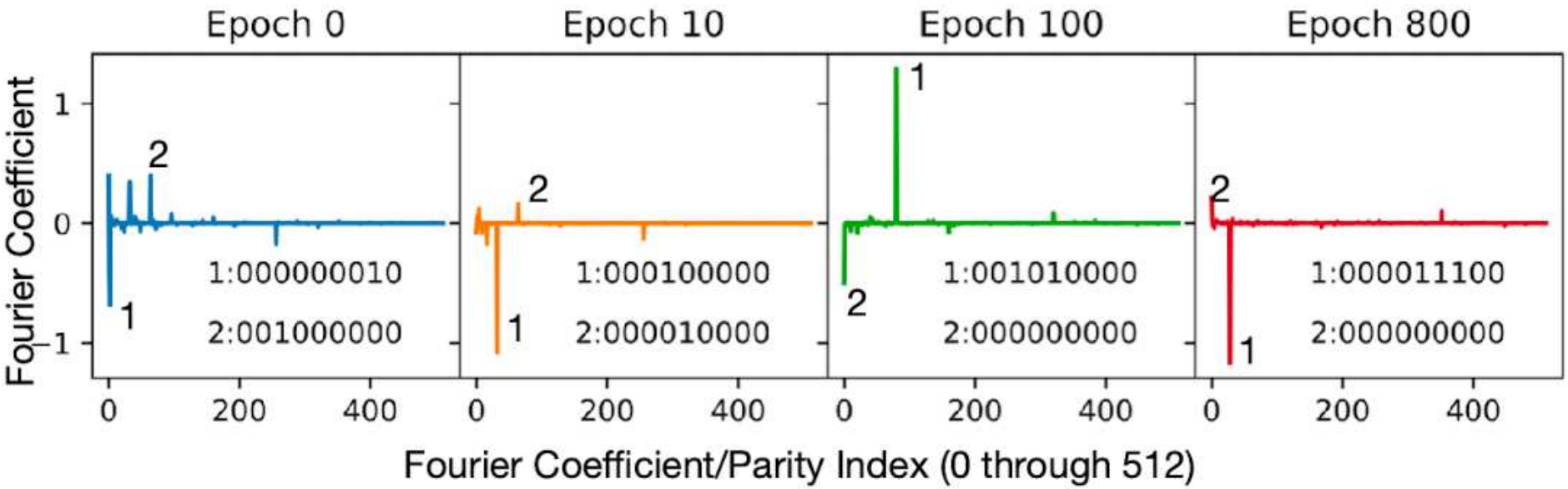
Goldreich-Levin Algorithm for general-purpose Approximate Conversion

- Good parity approximations can be found by looking at dominant terms in Fourier expansion of encoder
- Goldreich-Levin algorithm gives all FCs larger than a specified threshold with high probability.
  - This can be efficient if not many dominant coefficients.

Block #	Approx. sets for output 11	Min # queries	Approx. $\hat{f}(S)$
1	$\{u_1, u_2, u_3, u_4, u_5\}$	200	-0.81
2	$\{u_1, u_3, u_4, u_5\}$	25	1.0
3	Sol 1: $\{u_1, u_2, u_4\}$ Sol 2: $\{u_1, u_2, u_3, u_4\}$ Sol 3: $\{u_1, u_2, u_4, u_5\}$ Sol 4: $\{u_1, u_2, u_3, u_4, u_5\}$	800	0.4982 -0.5004 -0.5014 -0.4995
where	$u_1 = x_{i+2}$, $u_2 = x_{i+1}$, $u_3 = x_j$, $u_4 = x_{i-1}$, $u_5 = x_{i-2}$, $x =$ inputs		

Viewing Training from the Fourier Lens

- Since each constituent encoder can be viewed as a function $\{-1, 1\}^w \to \mathbb{R}$ for receptive window size w, we can look at its Fourier expansion
- During training, default initialization results in only degree 1 coefficients - *the NN can be represented as an $\mathbb{R}$-affine function.*
- As training progresses we see
  1. Emergence of a few dominant coefficients
  2. Progressive growth in degree of the dominant coefficients



Binary Cross-Entropy and its Decomposition

We can decompose cross-entropy into a KL-Divergence term and an entropy term

$$\begin{aligned}\mathbb{C}_j(\theta,\gamma) &:= \mathbb{E}[\mathrm{BCE}(u_j,\hat{u}_j)] \\ &= \mathbb{E}_Y\left[D_{KL}(\mathbb{P}[u_j|\mathbf{y}]||g_{\phi,j}(\mathbf{y}))\right] + \mathbb{H}(u_j|\mathbf{y})\end{aligned}$$

- $\mathbb{H}(u_j|\mathbf{y})$ depends only on the **encoder**!
- For a fixed encoder, $\mathbb{E}_Y\left[D_{KL}(\mathbb{P}[u_j|\mathbf{y}]||g_{\phi,j}(\mathbf{y}))\right]$ is minimized to 0 when $g_{\phi,j} = \mathbb{P}[u_j|\mathbf{y} = \cdot]$, i.e. when g_ϕ is the **MAP decoder**.

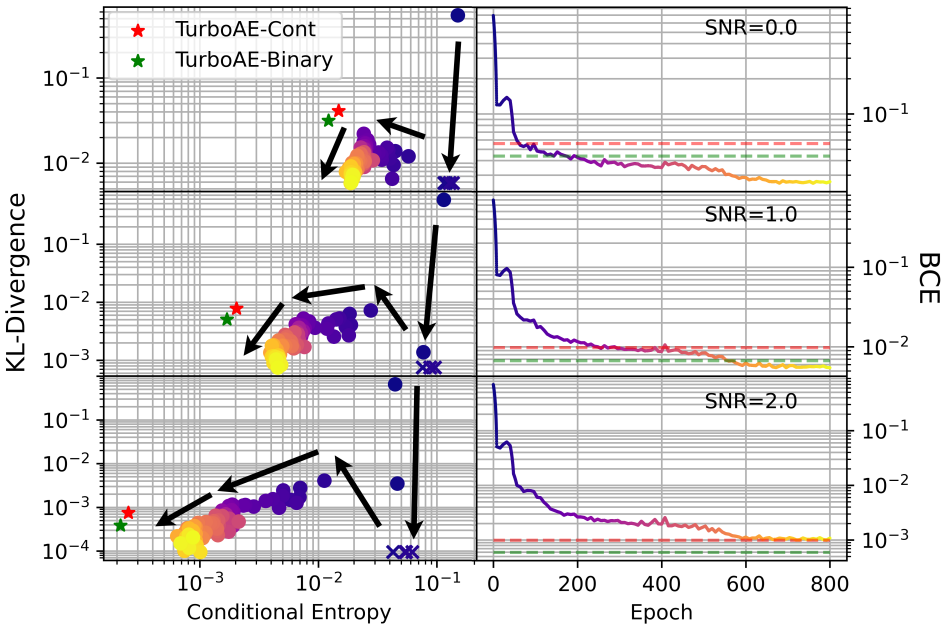
We can understand cross-entropy minimization as a back-and-forth between

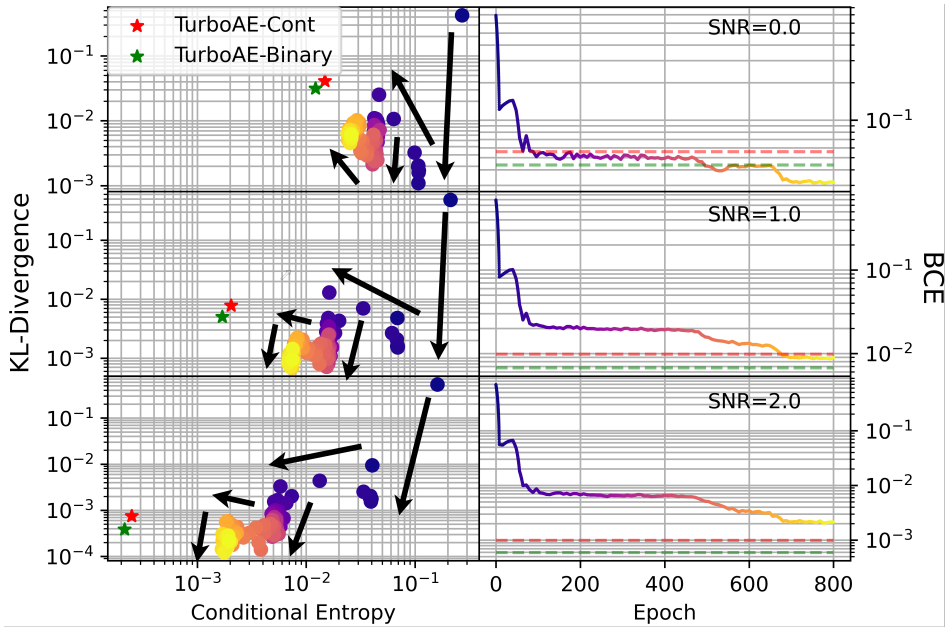
1. Finding an encoder
2. Finding a suitable decoder for the encoder

Feasible MAP decoding of TurboAE

- Decomposition helps us intuitively and analytically, but one step backwards computationally - difficult to estimate terms unlike BCE unless we can apply problem structure.
- For Turbo Codes, we can factor $\mathbb{P}[u_j|Y = \cdot]$ using the local dependence of the constituent convolutional codes.
- This allows us to use the Junction Tree algorithm for computation of $\mathbb{P}[u_j|Y = \cdot]$
  - For instances of TurboAE, only feasible for block lengths ≤ 40.
  - **Not** practical! Complexity scales like roughly $2^{k/2}$ for block length k.
  - Still useful for interpretation and study of TurboAE

Zig-zag Training of TurboAE





[1] Figures from *Mulgund et al., ISTC '23*

Conclusions and Further Work

- **Post-hoc interpretation** to improve efficiency and map back to known codes
- Studying **training** highlights parity attracting structure and difficulty in learning encoders
- Further work: provide *analytical results* to match *empirical observations* regarding the training process.

References I


N. Devroye, N. Mohammadi, A. Mulgund, H. Naik, R. Shekhar, Gy. Turán, Y. Wei, and M. Žefran, *Interpreting Deep-Learned Error-Correcting Codes*, ISIT, June 2022, pp. 2457–2462.


N. Devroye, A. Mulgund, R. Shekhar, Gy. Turán, M. Žefran, and Y. Zhou, *Interpreting Training Aspects of Deep-Learned Error-Correcting Codes – extended ArXiv version*, June 2023.


Yihan Jiang, Hyeji Kim, Himanshu Asnani, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath, *Turbo autoencoder: Deep learning based channel codes for point-to-point communication channels*, NIPS, December 2019, pp. 2758–2768.


Hyeji Kim, Yihan Jiang, Sreeram Kannan, Sewoong Oh, and Pramod Viswanath, *Deepcode: Feedback codes via deep learning*, IEEE Journal on Sel. Areas in Inf. Theory **1** (2020), no. 1, 194–206.

References II



A. Mulgund, N. Devroye, Gy. Turán, and M. Žefran, *Decomposing the Training of Deep Learned Turbo codes via a Feasible MAP Decoder*, 2023 12th International Symposium on Topics in Coding (ISTC), 2023, pp. 1–5.



Ashok Vardhan Makkuva, Xiyang Liu, Mohammad Vahid Jamali, Hessam Mahdavifar, Sewoong Oh, and Pramod Viswanath, *Ko codes: inventing nonlinear encoding and decoding for reliable wireless communication via deep-learning*, ICML (2021).



A. Mulgund, R. Shekhar, N. Devroye, Gy. Turán, and M. Žefran, *Evaluating interpretations of deep-learned error-correcting codes*, 2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton), September 2022, pp. 1–8.

References III


Emre Ozfatura, Yulin Shao, Alberto Perotti, Branislav Popović, and Deniz Gündüz, *All you need is feedback: Communication with block attention feedback codes*, IEEE Jour. on Sel. Areas in Info. Theory (2022), 1–1.